

Traditional UNIX-like operating systems have had an Access Control Mechanism that has remained, and still remains, one of the strongest security features of this family of operating systems.

We are all aware of the 'rwx' bits set on files and folders along with some special permissions. These permissions, along with user identity (UID) and group identity (GID), form the basis of traditional access control. This access control prevents unauthorised access to files and processes. *chroot* jails further confine filesystem access to a running process.

Is there a flaw in this age-old time-tested Access Control Mechanism? If no, what is the need for SELinux?

The inherent flaw in the traditional permissions model is that of DISCRETION. The owner of a particular file, for example, can change the permissions of an object, at will. Just imagine the following scenario: */etc/passwd*, the file that contains the user database, has default permissions of 644 and the file is owned by the root user, who—by accident or design—assigns it 666 permissions (global read and write). Any user could write into the user database, thus changing UIDs, home directories, etc. Imagine the security breach if the root user were to do such a thing.

The traditional Access Control Mechanism of permissions on files and processes is thus discretionary—it can be changed at the discretion of the owner and is often termed as Discretionary Access Control (DAC). Herein lies an inherent security flaw.

By contrast, SELinux implements Mandatory Access Control (MAC), where access control decisions are not at the discretion of individual users or even systems administrators.

Understanding MAC

Mandatory Access Control (MAC) ensures that the security policy of an organisation is adhered to. The various 'actors' in any security policy can broadly be classified as:

- Subjects
- Objects

Subjects perform certain operations on objects and the security policy specifies which of these operations are allowed or disallowed through Mandatory Access Control. In an operating system:

- Subjects are typically processes or threads
- Objects are typically files, directories, TCP/UDP ports, shared memory segments, network interfaces, etc

When an application process or thread tries to access objects that are disallowed in the security policy, an ACCESS DENIAL occurs, thus providing a layer of security between applications. Each application can be understood to be running in its domain, and subjects can typically access objects or perform operations within this domain, thereby confining the application within the boundaries of its 'domain'.

A new process/thread, on execution, might create its own domain through something referred to as domain transition. We will come to this in the later part of this series.

To identify subjects and objects and to determine what

access is allowed to whom, SELinux implements a MAC mechanism called Type Enforcement. This enables a granular control of the access mechanism over the operating system.

Type Enforcement and Security Contexts

Under Type Enforcement, certain attributes are applied to all objects and subjects. These attributes are termed as Security Contexts. Each process and file/directory/port on the system is assigned a Security Context based on which the Type Enforcement policy allows/disallows access.

Security Contexts are stored in Extended Attributes (xattrs) on an ext2/ext3 filesystem. A typical SELinux security context is of the form: *User Identity:Role:Type/Domain*

In multi-layer security and multi-category systems, two more attributes—sensitivity and category—are added. We will come to these later in the series.

Roles and users in the security context are present to support RBAC (Role-Based Access Control) features in SELinux. RBAC is to enable user privileges within the SELinux system. This too will be discussed later.

In this introductory article, our focus will be on SELinux Type Enforcement to give you a feel of SELinux, and thus we will primarily highlight the third attribute of the Security Context—Type/Domain.

Introduction to SELinux policies

The SELinux Type Enforcement Policy is based on rules that apply on Security Contexts.

Processes running with a certain Security Context are either allowed or disallowed to perform operations (access permissions—read/write/getattribute/setattribute, etc) on objects with certain Security Contexts.

The total compilation of these rules is called a SELinux policy. Red Hat Enterprise Linux comes with two standard policies—Targeted and Strict.

To briefly summarise the above:

- All processes (subjects), files, TCP/UDP ports, etc, (objects) are assigned a security context.
- Rules are created that allow/disallow access type (read/write, etc) by subjects to objects based on Security Contexts of the subject and that of the object.
- The set of these rules creates a policy. Currently, we are discussing Type Enforcement Policies.
- This policy is applied mandatorily to the system and cannot be overridden by users and even system administrators.
- Mandatory Access Control through Type Enforcement Policies enables SELinux fine-grained control over the operating system and any applications that might be running on top of it—adding an extra defence layer.

Enabling SELinux

Before beginning to explore SELinux, we need to enable it in the system. I use Red Hat Enterprise Linux 5 on my desktop, though the concepts and most of the commands would be the same on other flavours as well.